Degree & Branch	B.E. Computer Science & Engineering	**Semester**	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	**Batch: 2023-2028**	**Due date:**

Experiment 1: Working with Python Packages

Name: Mani Chidambaram

Aim

To explore Python libraries such as NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib, and apply machine learning workflows on five datasets (Iris, Loan, Diabetes, Spam, MNIST) to understand data handling, preprocessing, model training, and evaluation.

Libraries Used

The following Python libraries were central to this experiment:

- **Pandas:** For data manipulation and reading CSV files.
- **NumPy:** For numerical operations and creating dummy data.
- **Scikit-learn:** For machine learning models (Logistic Regression, Linear Regression, Random Forest, Gaussian Naive Bayes), data preprocessing (StandardScaler), feature selection (SelectKBest), and model evaluation metrics.
- **Matplotlib & Seaborn:** For data visualization, including pair plots, scatter plots, heatmaps, and bar charts.

1 Iris Dataset - Classification

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

iris = sns.load_dataset('iris')
sns.pairplot(iris, hue='species')
plt.show()

X = iris.drop('species', axis=1)
y = iris['species']
X_new = SelectKBest(score_func=f_classif, k='all').fit_transform(X, y)

X_train, X_test, y_train, y_test = train_test_split(X_new, y, test_size=0.2,
    random_state=42)
model = LogisticRegression(max_iter=200)
```

```python
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\n--- IRIS DATASET ---")
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Listing 1: Python code for Iris dataset classification.

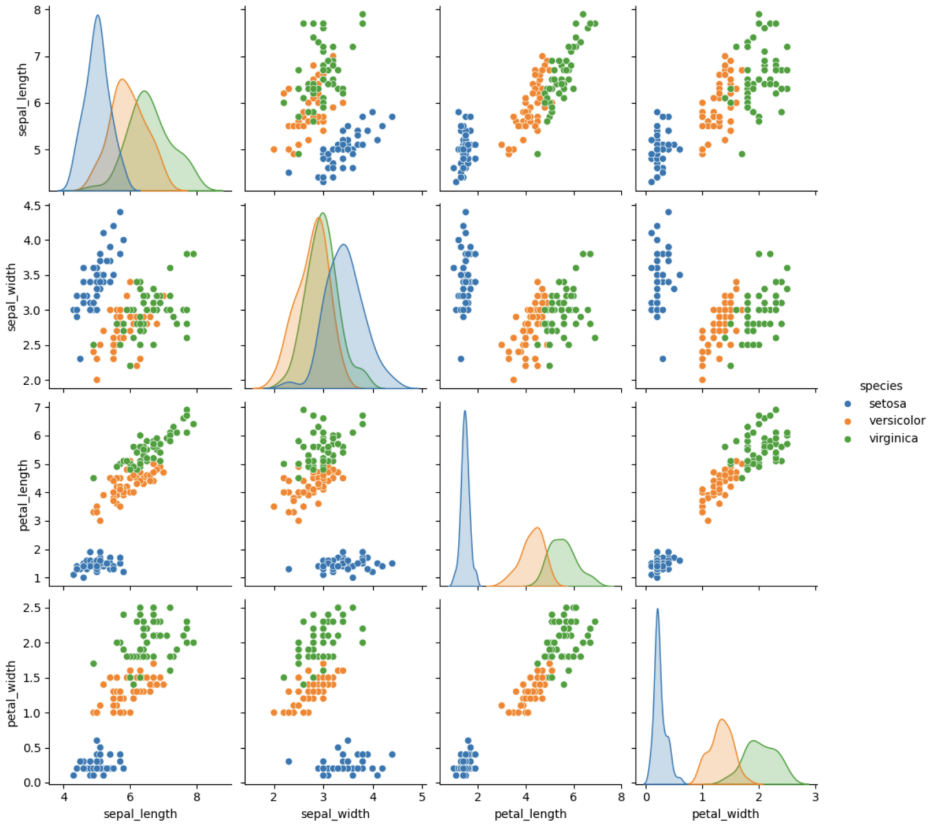

Figure 1: Pair plot of the Iris dataset, showing relationships between features, colored by species.

Result

The Logistic Regression model achieved **100% accuracy** on the test set. The precision, recall, and F1-score were 1.00 for all three species (setosa, versicolor, and virginica), indicating perfect classification.

2 Loan Amount Prediction - Regression

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df = pd.read_csv('/content/train.csv')

df.drop(columns=["Customer ID", "Name", "Property ID"], inplace=True)
df.dropna(inplace=True)

X = pd.get_dummies(df.drop(columns=["Loan Sanction Amount (USD)"]), drop_first=
    True)
y = df["Loan Sanction Amount (USD)"]

X = StandardScaler().fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("RMSE:", mean_squared_error(y_test, y_pred))
print("R-squared Score:", r2_score(y_test, y_pred))

plt.figure(figsize=(8,6))
sns.scatterplot(x=y_test, y=y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("Actual Loan Sanction Amount (USD)")
plt.ylabel("Predicted Loan Sanction Amount (USD)")
plt.title("Actual vs Predicted Loan Amounts")
plt.show()
\end{{lstlisting}

\begin{figure}[h!]
    \centering
    \includegraphics[width=0.7\textwidth]{images/loan_prediction.png}
    \caption{Actual vs. Predicted Loan Sanction Amounts.}
    \label{fig:loan}
\end{figure}

\subsection*{Result}
The Linear Regression model resulted in a Root Mean Squared Error (RMSE) of
    approximately 144,821 and an $R^2$ score of -0.015. The negative $R^2$ score
     suggests that the model performs worse than a horizontal line, indicating
    it failed to capture the underlying trends in the data.

\clearpage
%
    ================================================================================

% 3. DIABETES DATASET
%
    ================================================================================
```

```
\section{Diabetes Prediction - Classification}
\begin{lstlisting}[caption=Python code for Diabetes prediction using Random Forest.]
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, auc
# ... (data loading and splitting from your script) ...

df = pd.read_csv('diabetes.csv')
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_scaled = StandardScaler().fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.25, random_state=42)
clf = RandomForestClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

# ... (code for plotting confusion matrix, feature importances, and ROC curve) ...
```

Listing 2: Python code for Loan Amount prediction.

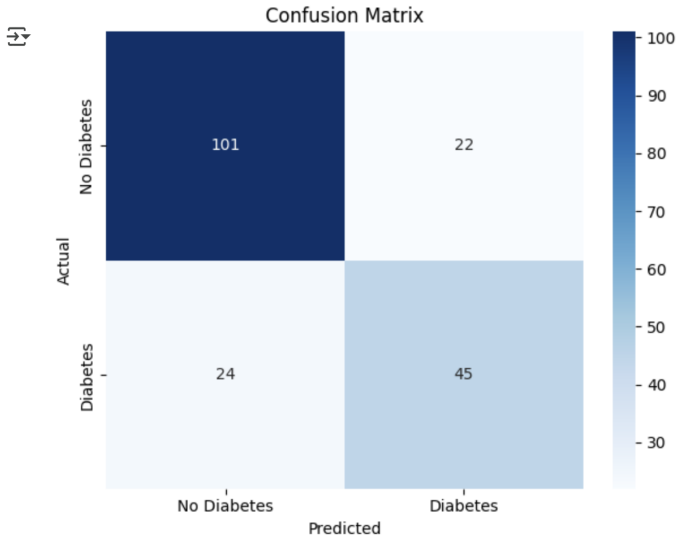

Figure 2: Confusion Matrix for Diabetes Prediction.

Result

The Random Forest model achieved an **accuracy of 72.4%**. The ROC curve shows an AUC (Area Under Curve) of 0.80, indicating a good measure of separability. Glucose, BMI, and Age were identified as the most important features for prediction.

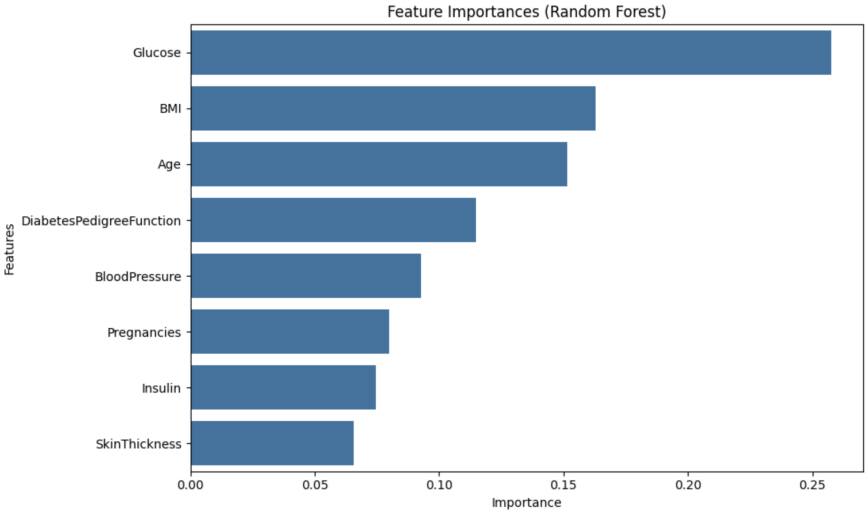

Figure 3: Feature Importances from the Random Forest model.

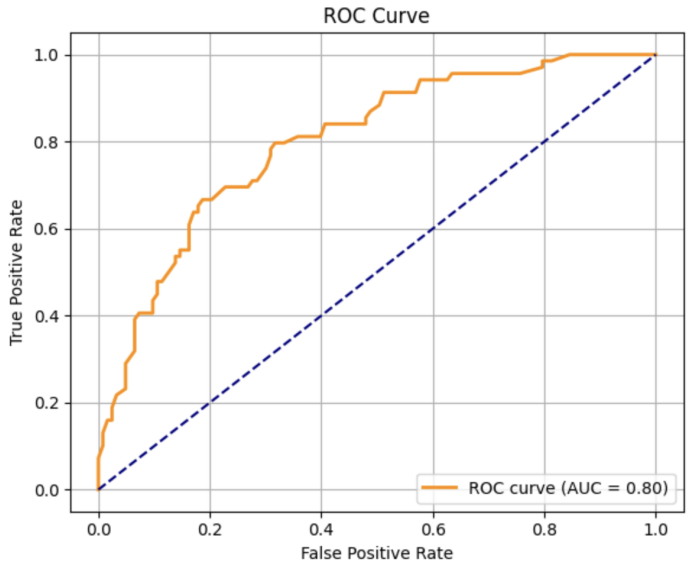

Figure 4: ROC Curve for the Diabetes classifier.

3 Spam Email Detection - Classification

```python
from sklearn.naive_bayes import GaussianNB
# ... (imports and data loading from your script) ...

df = pd.read_csv("spambase.data", header=None)
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=0)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Listing 3: Python code for Spam detection using Gaussian Naive Bayes.

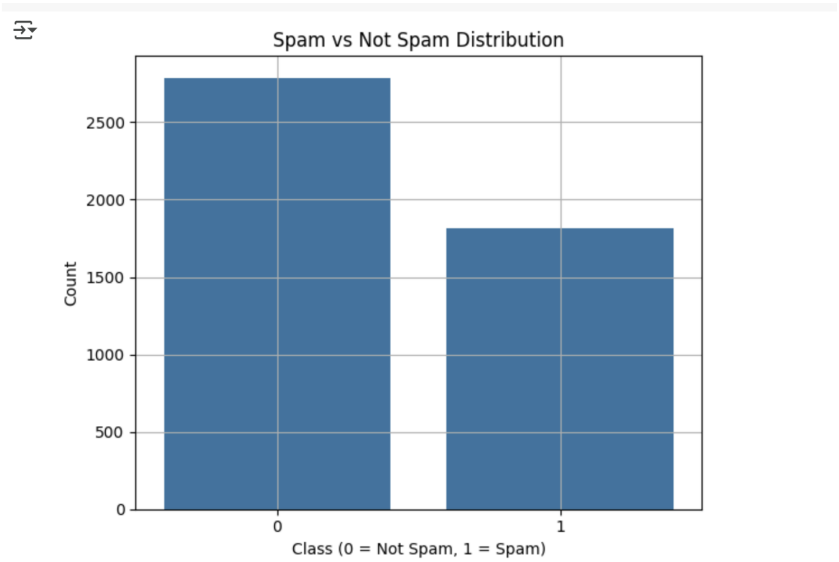

Figure 5: Distribution of Spam vs. Not Spam classes in the dataset.

Result

The Gaussian Naive Bayes classifier achieved an **accuracy of 82.2%**. The model showed strong recall (0.94) for detecting spam emails (class 1) but lower precision (0.72), indicating it correctly identifies most spam but also misclassifies some legitimate emails as spam.

4 MNIST Digits - Classification

```python
from sklearn.datasets import load_digits
# ... (imports and data loading from your script) ...

digits = load_digits()
X = digits.data
y = digits.target

plt.imshow(digits.images[0], cmap='gray')
plt.show()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)
model = LogisticRegression(max_iter=3000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Listing 4: Python code for MNIST digit classification.

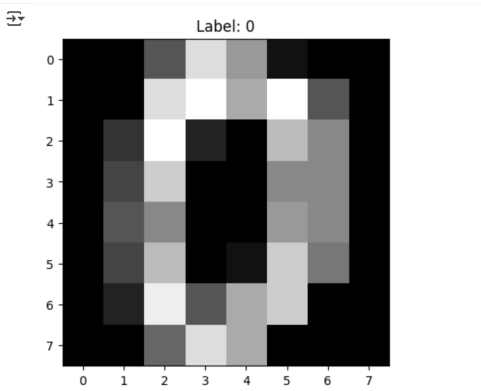

Figure 6: Sample image of a digit from the MNIST dataset.

Result

The Logistic Regression model achieved a high **accuracy of 96.5%** on the MNIST test set. The precision and recall were strong across all 10 digit classes, demonstrating the model's effectiveness in multi-class classification.

Results and Discussions

Dataset and Algorithm Summary

The experiments covered both regression and classification tasks using appropriate algorithms for each.

Table 1: Summary of Datasets and ML Tasks

Dataset	**ML Task**	**ML Type**	**Model Used**
Iris Dataset	Classification	Supervised	Logistic Regression
Loan Amount	Regression	Supervised	Linear Regression
Predicting Diabetes	Classification	Supervised	Random Forest
Email Spam	Classification	Supervised	Gaussian Naive Bayes
MNIST Digits	Classification	Supervised	Logistic Regression

Model Performance

Performance varied across datasets. The models for Iris and MNIST, which have well-separated classes, performed exceptionally well (96-100% accuracy). The Diabetes and Spam classification tasks were more challenging, yielding accuracies of 72% and 82% respectively. The Loan Amount regression model performed poorly, indicating that a simple linear model is not suitable for that dataset's complexity.

Learning Outcomes

- Gained practical experience in using core Python libraries for machine learning, including Pandas for data handling, Scikit-learn for modeling, and Matplotlib/Seaborn for visualization.
- Learned to implement a complete machine learning workflow: data loading, preprocessing (scaling, encoding), model training, prediction, and evaluation.
- Successfully applied both regression (Linear Regression) and classification (Logistic Regression, Random Forest, Naive Bayes) algorithms to diverse, real-world datasets.
- Developed an understanding of how to interpret key performance metrics such as accuracy, confusion matrix, precision, recall, F1-score for classification, and RMSE and R^2 score for regression.